

Compresser :

T_ListeChaine EST POINTEUR SUR T_Cellule

T_Cellule EST ENREGISTREMENT

Arbre : T_Arbre

Suivant : T_ListeChaine

T_Arbre EST POINTEUR SUR T_Branche

T_Branche EST ENREGISTREMENT

Symbole : Caractère

Fréquence : Entier

Code : T_Octet

Fils_Gauche : T_Arbre

Fils_Droit : T_Arbre

R0 : Compresser un fichier texte d'après le codage Huffman

R1 : Comment "Compresser un fichier texte d'après le codage Huffman" ?

Déterminer les fréquences d'apparition des caractères

Fichier_texte : **in str** ;
Liste_Feuille: **out T_ListeChaine**

Construire l'arbre de Huffman

Liste_Feuille: **in T_ListeChaine**
Arbre : **out T_Arbre**

Créer la table de codage

Coder le texte

R2 : Comment "Déterminer les fréquences d'apparition des caractères ?

Initialiser le tableau de fréquence

Tableau_freq : **out T_table_freq**

TantQue not Fin_Fichier **Alors**

Récupérer le caractère

Caractère : **out caractère**

Mettre à jour la fréquence

Tableau_freq : **in out T_table_freq**

FinTQ

R2 : Comment "Construire l'arbre de Huffman" ?

TantQue Longueur(Tableau_freq) > 1 **Faire**

Rechercher 2 plus faibles fréquences

Liste_Feuille: **in T_ListeChaine**
Branche1, Branche2 : **out T_Arbre**

Construire la nouvelle branche

Nouvelle_Branche : **out T_Branche**

Mettre à jour l'arbre de Huffman

FinTQ

Récupérer l'Arbre

Arbre : **out T_Arbre**

R2: Comment : "Créer la table de codage " ?

Fonction_Code(Arbre, Code)

Si FG = null

Arbre.Code = Code

Sinon

Fonction_Code(Arbre.FG, Code+'0')

Fonction_Code(Arbre.FD, Code+'1')

Fin Si

Fonction_dict(Arbre) return dictionnaire

Si Fg = null

return CoupleCleValeur(Symbole,Code)

Sinon

return assenmbagle(Fonction_dict(Arbre.FG) , Fonction_dict(Arbre.FD))

Fin Si

R2 : Comment : "Coder le texte" ?

Initialiser Texte_codé

TantQue not Fin_Fichier **Alors**

Récupère caractère

Récupère code huffman associé à caractère

in Caract , table_huffman code: out

Ajouter code à texte_codé

Sauver octet par octet

Fin TantQue

R3 : Comment "Mettre à jour la fréquence" ?

Curseur <- Tableau_freq

Répéter

Passer à l'élément suivant

Jusqu'à Curseur.all.Arbre.Symbole = Caractère or Curseur = Null

Si Curseur = Null Faire

Créer nouvel arbre

Nouvel_Arbre : **out T_Arbre**

Créer Nouvelle Cellule

Nouvel_Arbre : **in T_Arbre**

Nouvelle_Cellule : **out T_Cellule**

Sinon

Curseur.all.Arbre.Frequence <- Curseur.all.Arbre.Frequence + 1

FinSi

R3 : Comment "Rechercher 2 plus faibles fréquences" ?

Initialiser les variables de recherche des minimums

TantQue Curseur ≠ null Faire

Arbre_actuel <- Curseur.all.Arbre

Si Arbre_actuel.Frequence < Min2 Faire

Min2 <- Arbre_actuel.Frequence

Cellule2 <- Arbre_actuel

FinSi

Si Min1 > Min2 Faire

Inverser les 2 minimums

FinSi

Curseur <- Curseur.all.Suivant

Fin Tant Que

R3 : Comment "Construire la nouvelle branche" ?

Nouvelle_Branche : T_Branche

Nouvelle_Branche.all.Symbole <- '0'

Nouvelle_Branche.all.Frequence <- Min1 + Min2

Nouve_Branche.all.Code <- 0

Nouvelle_Branche.all.Fils_Gauche <- Branche1.all.Arbre

Nouvelle_Branche.all.Fils_Droit <- Branche2.all.Arbre

R3 : Comment "Mettre à jour l'arbre de Huffman" ?

Supprimer Cellule1 et Cellule2

Ajouter la nouvelle branche

R4 : Comment "Initialiser les variables de recherche des minimums" ?

Cellule1 <- Liste_Feuille

Min1 <- Branche1.all.Fréquence

Cellule2 <- Liste_Feuille.all.Suivant

Min2 <- Cellule2.all.Fréquence

Curseur <- Cellule2.all.Suivant

Si Min1 > Min2 Faire

Inverser les 2 minimums

FinSi

R4 : Comment "Créer un nouvel arbre" ?

Nouvel_Arbre.all.Symbole <- Caractère

Nouvel_Arbre.all.Frequence <- 1

Nouvel_Arbre.all.Fils_Gauche <- Null

Nouvel_Arbre.all.Fils_Droit <- Null

R4 : Comment "Créer nouvelle Cellule" ?

Nouvelle_Cellule.all.Suivant <- Null

Nouvelle_Cellule.all.Arbre <- Nouvel_Arbre

Curseur.all.Suivant <- Nouvelle_Cellule

R4 : Comment "Supprimer Cellule" ?

Si Liste_Feuille.Arbre = Cellule.Arbre Faire

Liste_Feuille <- Cellule.Suivant

Sinon

Parcours de la liste chaînée jusqu'à trouver Cellule avec précédent et suivant jusqu'à trouver cellule

Fin Si

Cas particulier

Précédent.Suivant <- Cellule.Suivant

Détruire(Cellule)

R4 : Comment "Ajouter nouvelle branche" ?

Nouvelle_Cellule.Suivant <- Liste_Feuille

Liste_Feuille <- Nouvelle_Cellule

R4 : Comment "récupérer l'arbre" ?

Arbre <- Liste_Feuille.Arbre

Décompresser

Type Element

Type Element_pointeur EST POINTEUR SUR Element

Type Element EST ENREGISTREMENT

Symbole : Chr

Suivant : Element_pointeur

R0 : Décompresser un texte codé à l'aide de l'arbre de Huffman

R1 : Comment « Décompresser un texte codé à l'aide de l'arbre de Huffman » ?

Lire le texte codé

Fichier : **in Ada.Text_IO.File_Type**

Récupérer et stocker les symboles

Liste_Symboles : **in out Element_pointeur**

Curseur : **out T_ListeChaine**

Bit : **in out Integer**

Symbole : **in out Character**

Afficher le texte décodé

Liste_Symboles : **in**

R2 : Comment « Lire le texte codé » ?

Ouvrir le fichier compresser et lire son contenu

R2 : Comment « Récupérer et stocker les symboles »

Créer une liste vide Liste_Symboles pour stocker le texte décodé

Tant Que Symbole /= « /\$ » Faire

Parcourir chaque séquence de bit entre les deux points et remonter l'arbre de Huffman en fonction

Ajouter à Liste_Symboles le symbole correspondant au nœud atteint quand la séquence est terminée

Fin TantQue

R2 : Comment « Afficher le texte décodé » ?

```
Curseur <- Liste_Symboles  
Tant Que Curseur.all.Suivant /= Null Faire  
    Afficher (Curseur.all.Symbole)  
Fin TantQue
```

R3 : Comment “ Ouvrir le fichier et lire son contenu ”?

```
Ada.Text_IO.Open (File => Fichier, Mode => Ada.Text_IO.In_File, Name => " exemple-sujet.txt.hff")
```

R3 : Comment « Créer une liste vide Liste_Symboles pour stocker le texte décodé » ?

```
Liste_Symboles = Null
```

R3 : Comment « Parcourir chaque séquence de bit entre deux points et remonter l'arbre de Huffman en fonction » ?

```
Curseur <- Liste_Feuille.Arbre  
Tant que Bit = 0 ou Bit =1 Faire  
    Parcourir l'arbre  
Fin Tant Que  
Symbole <- Curseur.all.Arbre.Symbole  
Curseur <- Curseur.all.Suivant
```

R3 : Comment « Ajouter à Liste_Symboles le symbole correspondant au nœud atteint quand la séquence est terminée » ?

```
Nouvel_Element.Symbole <- Symbole  
Nouvel_Element.Suivant <- Liste_Symboles  
Liste_Symboles <- New_Element
```

R4 : Comment « Parcourir l'arbre » ?

Si Bit = 0 Alors

Descendre la branche de gauche

Sinon Si Bit = 1

Descendre la branche de droite

Fin Si

Curseur <- Curseur.all.Suivant

R5 : Comment « Descendre la branche de gauche » ?

R5 : Comment « Descendre la branche de droite » ?

	Règle	Eval uati on (I/P/ A/+)	Justification/com mentaire (optionnel)	Inacceptable (I)	Partiellement atteint (P)	Atteint (A)
For me	Respect de la syntaxe Ri : Comment "... une action complexe ..." ? des actions combinées avec des structures de controle Rj : ...	A		Pas de numéro de raffinage (R0, R1, R2...) OU Pas de "Comment" OU Pas de rappel de l'action/expression complexe OU Pas de décomposition ou elle est mal indentée OU les Ri sont mentionné dans l'algo complet (non séparés)	l'action/expression est donnée ET la décomposition est donnée MAIS Erreur sur le numéro (niveau) de raffinage OU Pas de décalage pour les actions	Le bon numéro pour le raffinage ET le "Comment" est là ET l'action/complexe est donnée ET la décomposition est donnée indentée ET les actions sont décalées
	Verbe à l'infinitif pour les actions complexes	A		Pas ou peu de verbes à l'infinitif (moins de 20 %)	Moins de 70 % de verbes à l'infinitif	Tous les verbes sont à l'infinitif
	Nom ou équivalent pour expressions complexes	A		Utilisation de verbes à l'infinitif pour nommer une expression complexe	Moins de 10 % d'expressions complexes mal nommées	Aucun verbe à l'infinitif pour nommer une expression complexe
	Tous les Ri sont écrits contre la marge et espacés	A		Les Ri sont décalés en fonction de la valeur de i ET il n'y a pas de ligne blanche entre eux	Les Ri sont bien contre la marge OU 1 ou 2 lignes blanches séparent les Ri	Tous les Ri sont contre la marge ET 1 ou 2 lignes blanches les séparent
	Les flots de données sont définis	A		Pas de flot de données	Le flot de données est donné pour quelques actions ou expressions complexes OU les in et out ne sont pas donnés OU le type n'est pas donné lors de la première apparition d'une donnée	Les flots de données sont donnés pour presque toutes les actions ou expressions complexes ET Les in et les out sont présents ET le type d'une donnée est défini à sa première apparition
	Une seule structure de contrôle par raffinage	P		3 ou plus structures de contrôle (conditionnelle ou répétition) dans un raffinage	2 décisions ou répétitions au plus par raffinage	au plus 1 décision ou répétition par raffinage
	Pas trop d'actions dans un raffinage (moins de 6)	P		Beaucoup trop d'actions : > 9	Trop entre 6 et 9	Moins de 6
	Bonne présentation des structures de contrôle	A		Plus de 3 structures de contrôle sont mal présentées	1 ou 2 structures de contrôle sont mal présentées	Toutes les structures de contrôle sont présentées comme en cours

Fon d	Le vocabulaire est précis	P		On ne comprend pas plusieurs noms utilisés : pas significatifs OU utilisation d'abréviations OU mélange de plusieurs langues	On ne comprend pas quelques noms.	On comprend tous les noms ET pas d'abréviations ET pas de mélange de plusieurs langues
	Le raffinage d'une action décrit complètement cette action	P		Pour au moins 2 décompositions (raffinages) : Il manque des actions OU il manque des structures de contrôle (répétition, décision) OU il y a des actions interdites (comme Recommencer, Continuer en...)	Idem mais pour 1 décomposition seulement	Toutes les décompositions respectent les trois conditions.
	Le raffinage d'une action ne décrit que cette action	P		Pour au moins 2 décompositions : Il y a des actions en trop	Idem mais pour 1 décomposition seulement	Aucune action en trop dans aucune décomposition
	Les flots de données sont cohérents	P		Au moins 2 fois, on constate : des in avant des out pour des données locales OU des out sur des données en in de l'action complexe OU pas de out sur des données en out de l'action complexe OU des actions ne sont pas dans l'ordre vis à vis du flot	Idem mais on le constate 1 fois seulement	Les 4 règles sont toujours respectées
	Pas de structure de contrôle déguisée	A		Au moins 2 structures de contrôle déguisées (Faire ... si ..., Incrémenter tant que... calculer selon ...)	1 structure de contrôle déguisée	Aucune structure de contrôle déguisée
	Qualité des actions complexes	P		Au moins 2 fois, certaines actions ne sont pas explicites (i.e. on ne peut pas exécuter mentalement sans regarder le raffinage de certaines de ces actions) OU des actions pourraient être regroupées en une action complexe	Idem mais sur 1 raffinage seulement	Toujours respecté